



使用样式定制控件外观

北京理工大学计算机学院
金旭亮

概述



在Web前端设计领域，网页的样式是由样式表控制的，独立于特定的HTML网页，这种设计理念深入人心，并且在实践中取得了很好的效果。JavaFX也采用了这种思想，使用样式表来确定UI控件的显示效果。



样式表中包含可以控制用户界面元素外观的样式定义。在JavaFX程序中使用CSS与在HTML中使用CSS类似。JavaFX中的CSS建立在W3C CSS 2.1版（参见<http://www.w3.org/TR/CSS21>）的基础之上，在其中加入了3.0版的部分内容和一些支持特定JavaFX特性的扩展内容。



JavaFX为所有的控件都内置了一份默认的样式，以样式类的方式提供，这些样式类，其名字为对应的JavaFX控件的类名（转为小写），比如按钮（Button）所对应的样式类为“.button”，通过给这个样式类定义自己的样式规则，可以修改默认样式。

JavaFX样式的工作方式



与CSS类似，JavaFX样式表也有“选择器 (Selector)”的概念，先使用“**id选择器 (#)**”和“**类选择器 (.)**”来选择UI控件，再为其指定样式。



JavaFX专用的样式属性，都以“-fx-”打头。

```
.custom-button {  
    -fx-font: 16px "Serif";  
    -fx-padding: 10;  
    -fx-background-color: #CCFF99;  
}
```

自定义样式



可以创建一个或多个样式表来覆盖默认的样式或者是添加自己的样式。一般来说，创建的样式表文件都应该以.css为后缀，推荐将其放置到资源文件夹中与控制器或视图包名一致的文件夹下，这样可以简化其装载代码。



可以使用Scene对象的getStylesheets()方法加载一个或多个样式表文件：

```
Scene scene = new Scene(.....);  
//从资源文件夹中加载样式表文件，这里假设它放在与包名一致的资源文件夹下  
var cssFilePath = getClass().getResource("style.css").toString()  
//加载样式表资源  
scene.getStylesheets().add(cssFilePath);
```

设置控件的样式



当Scene调用getStylesheets()方法加载样式表文件之后，定义于样式表中的样式，可以作用于Scene所关联的“场景图(Scene Graph)”中的所有节点。



特定的控件节点，可以调用它的getStyleClass()方法，加载特定的样式类：

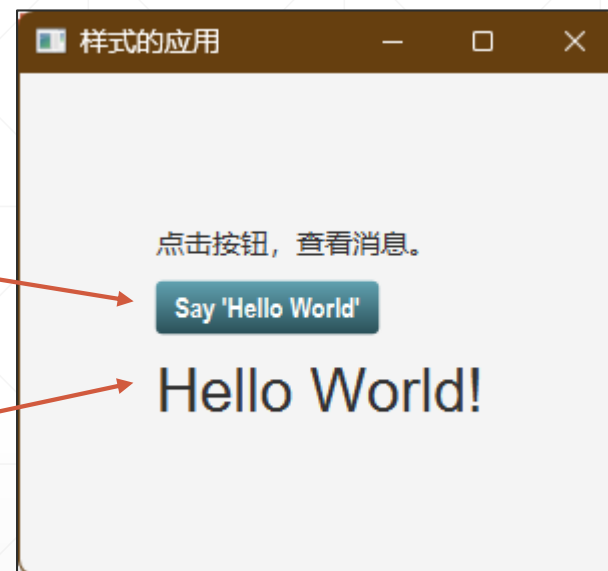
```
HBox hbox = new HBox();  
hbox.getStyleClass().addAll("样式类1", "样式类2");
```

样式如何影响控件的显示效果

直接修改默认按钮控件
的样式

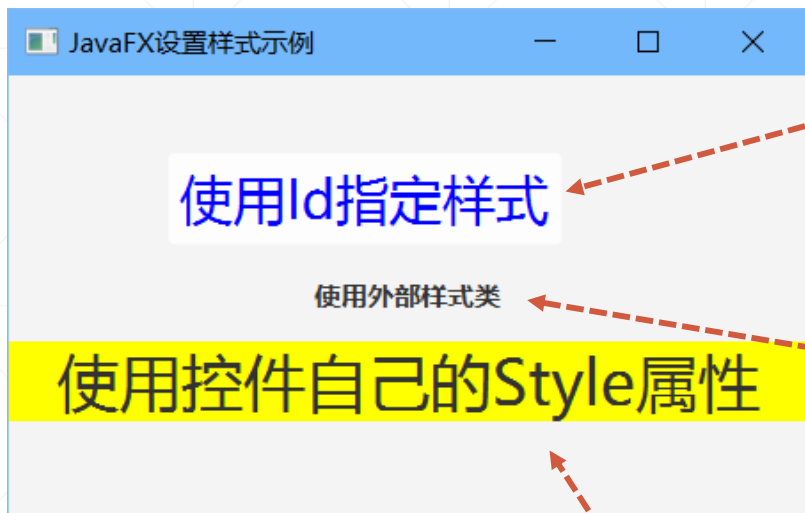
```
- .button {  
    -fx-text-fill:white;  
    -fx-font-family:"Arial Narrow";  
    -fx-font-weight:bold;  
    -fx-background-color:linear-gradient(#61a2b1,#2A5058);  
}  
  
- #textstyle {  
    -fx-font: 30px "Arial";  
    -fx-fill:white;  
}
```

为指定id的控件设置样式



示例：ButtonAndStyle.java

在FXML中设置样式



StyleRuleWithFxml.java

```
<Button fx:id="clickMe" layoutX="90.0" layoutY="44.0"  
mnemonicParsing="false" text="使用Id指定样式" />
```

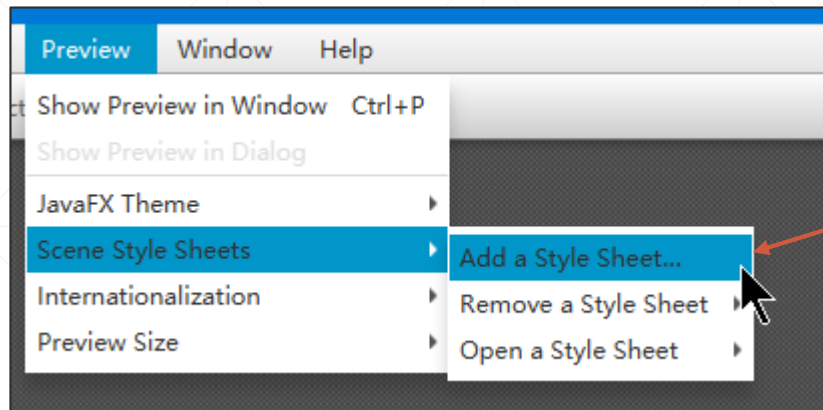
```
<Label fx:id="messageLabel" alignment="CENTER"  
layoutX="144.0" layoutY="114.0" styleClass="myClass"  
text="使用外部样式类" AnchorPane.leftAnchor="0.0"  
AnchorPane.rightAnchor="0.0" />
```

```
<Label alignment="CENTER" layoutX="71.0" layoutY="150.0"  
style="-fx-background-color: yellow; -fx-font-size: 35;"  
text="使用控件自己的Style属性"  
AnchorPane.leftAnchor="0.0" AnchorPane.rightAnchor="0.0" />
```



在FXML中设置样式，要比使用代码简洁方便，但要注意样式的覆盖与优先级问题。

在Scene Builder中预览样式



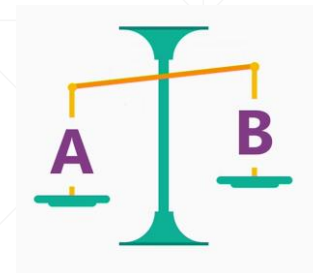
使用Scene Builder打开FXML文件之后，可以使用Preview菜单中的Scene Style Sheets子菜单加载外部样式表。

```
application.css
1 #btnTest{
2     -fx-font-size:30px;
3     -fx-text-fill: blue;
4     -fx-padding: 6 6 6 6;
5     -fx-border-radius: 8;
6 }
```



加载了外部样式表后Button，其显示效果已更换。

为特定控件指定样式的两种方式



在FXML中使用内联样式:

```
<Button text="Click me!">  
  <style>  
    -fx-padding: 10;  
    -fx-border-width: 3;  
  </style>  
</Button>
```

或者使用代码:

```
String styles =  
    "-fx-background-color: #0000ff;" +  
    "-fx-border-color: #ff0000;" ;  
  
Button button = new Button("Button 2");  
button.setStyle(styles);
```

样式的继承



施加于Parent节点的样式，会被其子节点所继承。

```
Button button1 = new Button("Button 1");  
Button button2 = new Button("Button 2");  
  
VBox vbox = new VBox(button1, button2);  
  
vbox.getStylesheets().add("button-styles.css");
```



JavaFX为Scene Graph的根控件提供了一个名为“root”的样式类，可以使用它来为整棵控件树中的节点定义通用样式规则。

.root样式类

```
root.css x
1  .root {
2      -fx-font-size: 20;
3      -fx-cursor: hand;
4      -fx-spacing: 10;
5      -fx-padding: 20;
6      /*自定义一个look-up属性值*/
7      -my-button-color: red;
8  }
9
10 .button {
11     -fx-text-fill: -my-button-color;
12 }
```

RootClassTest.java

```
public void start(Stage stage) {
    Label nameLbl = new Label(s: "姓名:");
    TextField nameTf = new TextField(s: "");
    Button closeBtn = new Button(s: "关闭");
    HBox root = new HBox();
    root.getChildren().addAll(nameLbl, nameTf, closeBtn);
    Scene scene = new Scene(root, v: 400, v1: 200);
    var url = getClass().getResource(name: "root.css").toExternalForm();
    scene.getStylesheets().add(url);
    stage.setScene(scene);
    stage.setTitle("Using the root Style Class Selector");
    stage.show();
}
```



样式表的“覆盖”



StyleRules.java

```
.button{  
    -fx-text-fill:white;  
    -fx-font-family:"Arial Narrow";  
    -fx-font-weight:bold;  
    -fx-background-color:linear-gradient(#61a2b1,#2A5058);  
    -fx-background-radius: 5;  
    -fx-font-size: 24px;  
}
```

```
.mybutton{  
    -fx-background-color: #2f4f4f;  
    -fx-alignment: center;  
    -fx-font:18px "Cooper Black";  
}
```



自定义样式类，优先于默认样式类。

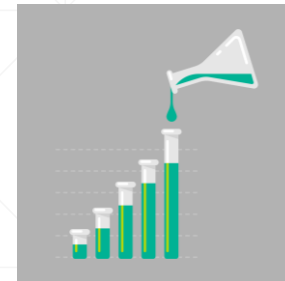
基于样式查找节点



```
Button b1 = new Button("Close");
b1.setId("closeBtn");
VBox root = new VBox();
root.setId("myvbox");
root.getChildren().addAll(b1);
Scene scene = new Scene(root, 200, 300);
...
Node n1 = scene.lookup("#closeBtn");
Node n2 = root.lookup("#closeBtn");
Node n3 = b1.lookup("#closeBtn");
Node n4 = root.lookup("#myvbox");
Node n5 = b1.lookup("#myvbox");
Set<Node> s = root.lookupAll("#closeBtn");
```

CSS选择器可以用于节点选择，只需把选择器传给Node类所定义的lookup/lookupAll方法即可。

小结：样式生效的优先级



使用代码在控件呈现后动态设置的样式

内联样式

父节点样式

root样式



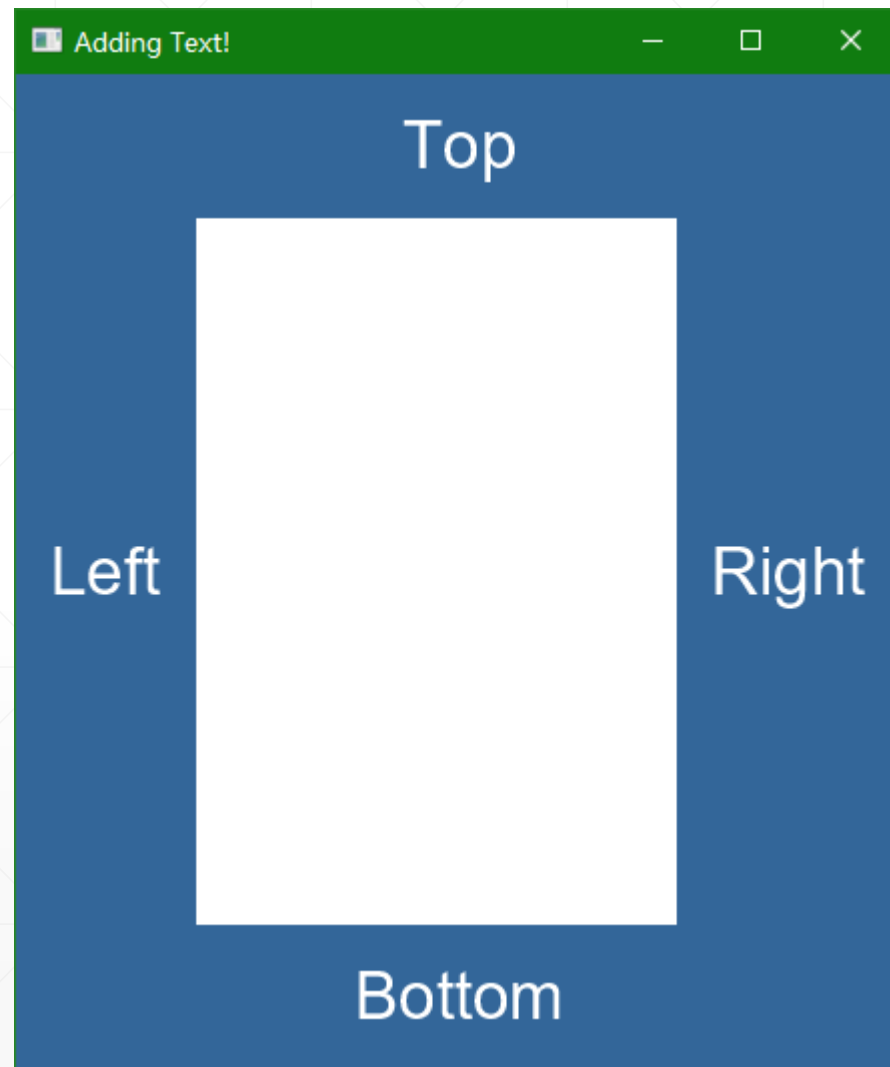
高

低



样式应用练习

使用BorderPane，配合样式表，实现右侧布局



参考答案: AddTextWithStyle.java